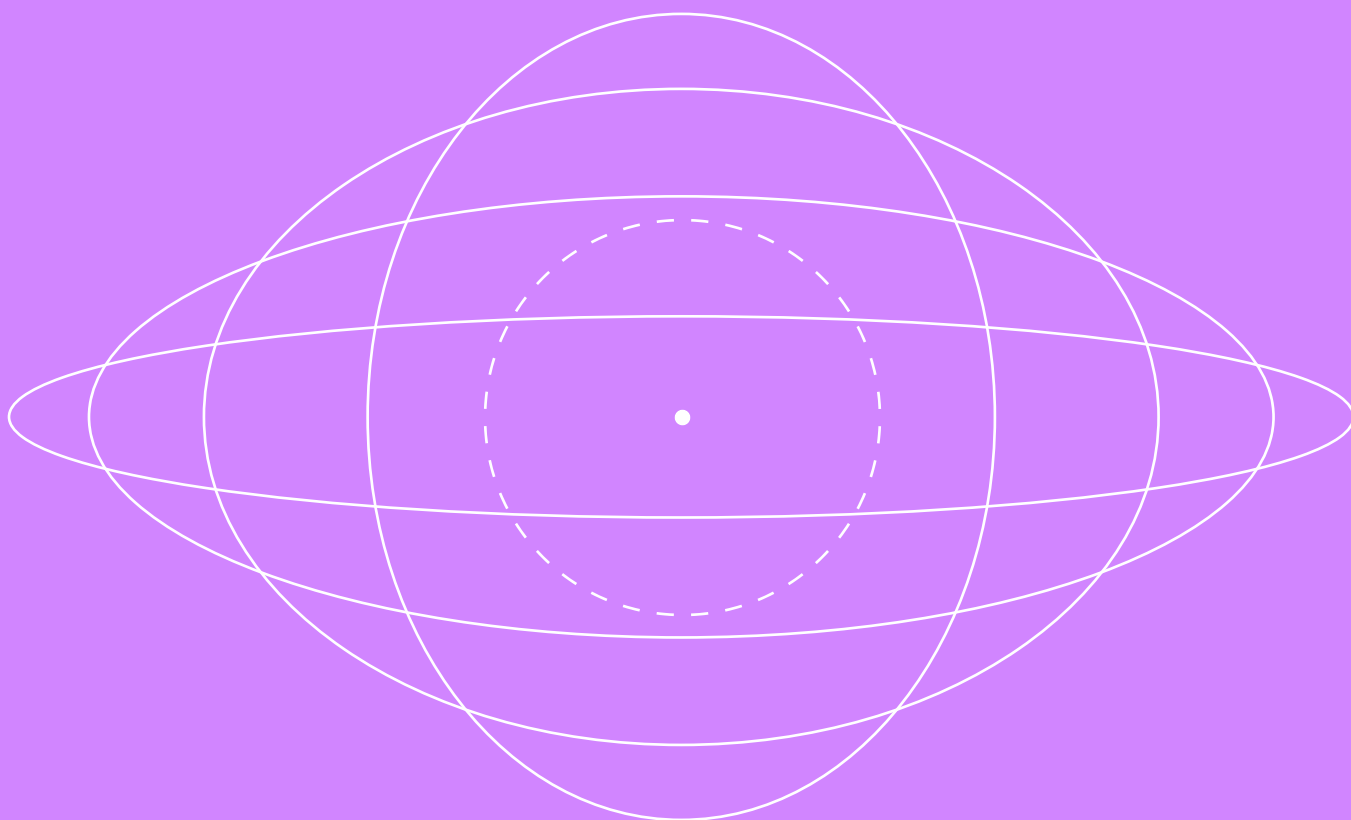


Многоверсионность. Снимки данных. Очистка и автоочистка



SQL И БАЗЫ ДАННЫХ (ДЛЯ РАЗРАБОТЧИКОВ)

НЕДЕЛЯ 13

Многоверсионность (MVCC) предоставляет высокую пропускную способность за счёт сохранения старых версий строк. Каждая версия маркируется идентификаторами транзакций `xmin` и `xmax`. В PostgreSQL тип счётчика транзакций (XID) имеет размер 32 бита. Это накладывает математическое ограничение в 4,2 миллиарда уникальных значений. Использование 64-битного счётчика увеличило бы размер заголовка каждой строки на 8 байт. В масштабах таблиц с миллиардами записей это приводит к расходу оперативной памяти и дискового пространства. Поэтому архитектурно был выбран 32-битный тип, что влечёт за собой необходимость управления переполнением счётчика.

Пространство идентификаторов транзакций организовано в виде кольцевого буфера. В системе отсутствует понятие абсолютного времени. Определение того, какая транзакция предшествовала другой, вычисляется с помощью модульной арифметики: $(XID_A - XID_B) \bmod 232$. Пространство в 4,2 миллиарда делится пополам. Для любой текущей транзакции 2 миллиарда значений XID считаются «прошлым» (эти транзакции завершены, их изменения видимы), а оставшиеся 2 миллиарда — «будущим» (созданные ими версии невидимы).

Данная кольцевая архитектура порождает проблему переполнения счётчика (XID Wraparound). При достижении физического предела в 4,2 миллиарда счётчик обнуляется и начинает отсчёт заново. Транзакция с идентификатором 100, создавшая строку в прошлом, после переполнения математически вычисляется как находящаяся в «будущем» относительно новой транзакции с идентификатором 2 000 000 100. Это приводит к тому, что старые, зафиксированные данные перестают удовлетворять правилам видимости снимков. Данные физически присутствуют на носителе, но логически исключаются из любой выборки. Данный феномен классифицируется как потеря данных.

Для предотвращения сценария потери данных PostgreSQL реализует механизм принудительной остановки (XID Stop Limit). Система вычисляет дистанцию между самым старым активным XID в базе данных и текущим выдаваемым XID. За один миллион транзакций до расчётной точки переполнения ядро СУБД генерирует ошибку `database is not accepting commands to avoid wraparound data loss` и переводит кластер в режим «только чтение» (read-only). Приём новых транзакций, модифицирующих данные, блокируется. Восстановление работоспособности требует ручного запуска процедуры вакуумирования в однопользовательском режиме (single-user mode), что влечёт за собой недоступность сервиса на запись.

Решением проблемы переполнения является механизм заморозки — VACUUM FREEZE.

В систему введён системный маркер `FrozenTransactionId`, равный двум. На уровне исходного кода СУБД этот идентификатор игнорирует правила модульной арифметики и оценивается как строго предшествующий (более старый) любому нормальному XID. Процесс очистки сканирует страницы данных и заменяет старые значения `xmin` на маркер заморозки. Метрикой контроля выступает поле `relfrozenxid` в системном каталоге `pg_class`. Оно фиксирует самый старый незамороженный XID в таблице, позволяя отслеживать дистанцию до точки переполнения.

Процесс очистки начинается с определения мёртвых кортежей. Мёртвые кортежи — это старые версии строк, которые больше не видны ни одному активному снимку данных. Наличие длительных транзакций блокирует этот процесс, так как старые версии строк остаются видимыми, что приводит к накоплению мусора и раздуванию физических файлов таблиц и индексов.

Глобальный алгоритм фоновой очистки состоит из нескольких фаз. Базовый запуск этого процесса сканирует таблицы и помечает мёртвые строки как свободное место. Данное пространство не возвращается операционной системе, а размечается внутри файла для повторного использования будущими операциями обновления и вставки.

1. На первой фазе происходит сканирование кучи. Система последовательно читает страницы табличного файла, идентифицирует мёртвые кортежи и собирает их физические адреса в специальный массив в оперативной памяти. Физического удаления данных на этом шаге не происходит. Размер массива для хранения адресов строго ограничен переменной `maintenance_work_mem`. Если количество мёртвых строк превышает выделенную память, сканирование таблицы приостанавливается. Система вынуждена переходить к очистке индексов частями, что приводит к многократным проходам и кратному росту нагрузки на дисковую подсистему.
2. Вторая фаза заключается в очистке индексов. База данных обходит все привязанные индексы и удаляет ссылки на собранные адреса мёртвых кортежей. Эта операция требует случайного чтения с диска, что делает её наиболее ресурсоёмкой частью всего алгоритма. Избыточное количество индексов на таблице напрямую замедляет процесс сборки мусора.
3. На третьей фазе происходит повторный проход по таблице для физической очистки страниц данных. Указатели строк переводятся в неиспользуемое состояние. Оставшиеся актуальные кортежи сдвигаются внутри страницы для дефрагментации и консолидации свободного дискового пространства в непрерывный блок.
4. Завершающая фаза включает обновление системных метаданных. Записывается актуальная статистика количества строк для планировщика запросов, а также обновляются карта свободного пространства и карта видимости. Физическое усечение файла и возврат дискового пространства операционной системе выполняются исключительно в том случае, если полностью пустые страницы образовались в самом конце табличного файла. В остальных ситуациях размер файла не уменьшается.

При выполнении операции вставки системе необходимо найти страницу с достаточным объёмом свободного места. Последовательное сканирование основного файла таблицы является неэффективным. Для решения этой задачи используется дополнительная структура данных — форк карты свободного пространства (FSM). Данный файл организован в виде N-арного дерева, где каждый узел хранит информацию о максимальном объёме доступного места в дочерних узлах. Подобная архитектура позволяет находить подходящую страницу для записи за логарифмическое время или определять необходимость выделения нового дискового блока в конце файла.

Вторым механизмом оптимизации ввода-вывода выступает карта видимости (VM). Она представляет собой компактный битовый массив, хранящийся в отдельном файле. На каждую страницу данных выделяется ровно два бита. Установка первого бита означает, что на странице отсутствуют мёртвые строки, а все транзакции, создавшие текущие версии записей, успешно зафиксированы. Установка второго бита сигнализирует о том, что все строки на странице заморожены для защиты от переполнения счётчика транзакций.

В архитектуре многоверсионности индексы не хранят информацию об идентификаторах транзакций. По умолчанию при обнаружении нужного ключа в индексе база данных обязана обратиться к основной таблице для проверки видимости конкретной версии строки. Наличие карты видимости позволяет обходить это ограничение. Если планировщик запросов определяет по битовой карте, что целевая страница является полностью видимой, обращение к основной таблице исключается. Данные извлекаются напрямую из индекса, что приводит к снижению количества операций дискового чтения.

Бит полной видимости устанавливается исключительно процессом очистки. Никакой другой процесс в базе данных не изменяет этот статус. При недостаточной частоте выполнения фоновой очистки высоконагруженных таблиц бит видимости сбрасывается при первых операциях обновления. Это приводит к деградации производительности читающих запросов, так как система возвращается к выполнению дополнительных операций случайного чтения из основного файла таблицы для проверки статуса строк.

В классической архитектуре многоверсионности операция обновления строки фактически представляет собой вставку новой версии записи и пометку старой версии как удалённой. Новая версия получает новый физический адрес. В результате система обязана обновить все индексы таблицы, чтобы они указывали на новый адрес, даже если изменяемое поле не участвует ни в одном из индексов. Это явление называется амплификацией записи и приводит к избыточному росту дисковых операций.

Для решения этой проблемы применяется механизм HOT. Его активация требует выполнения двух строгих условий. Во-первых, обновляемые столбцы не должны входить ни в один индекс таблицы. Во-вторых, на текущей странице данных должно присутствовать свободное физическое пространство для размещения новой версии строки. При соблюдении данных условий новая запись сохраняется на той же самой странице, а индексы не затрагиваются.

Механика работы основана на перенаправлении указателей внутри страницы. Индекс в базе данных ссылается на фиксированный указатель строки. При оптимизированном обновлении система модифицирует старый указатель, превращая его в адрес перенаправления. Индекс продолжает ссылаться на первоначальный указатель, который транслирует запрос к актуальной версии записи. Вся навигация по цепочке версий происходит строго в границах одного дискового блока, что исключает дополнительные операции чтения.

Данная архитектура обеспечивает дефрагментацию и самоочистку данных на лету. Когда процесс выполняет чтение страницы с цепочками перенаправлений, он параллельно оценивает статус старых версий строк. Если выявляются мёртвые записи, невидимые ни одной активной транзакции, читающий процесс самостоятельно удаляет их из структуры страницы и высвобождает дисковое пространство без ожидания фоновой очистки.

Эффективность механизма напрямую зависит от физического проектирования таблиц и настройки параметра `fillfactor`. Стандартное поведение системы подразумевает стопроцентное заполнение страниц при вставке данных, что исключает возможность локального размещения новых версий при последующих обновлениях. Установка параметра `fillfactor` в значение ниже максимального, например 90, заставляет подсистему хранения резервировать заданный процент свободного места на каждой новой странице. Заблаговременное выделение дискового пространства гарантирует локализацию обновляемых строк и кратное снижение нагрузки на систему ввода-вывода.

Процесс автоматической очистки управляется процессом-координатором `autovacuum launcher`. Данный процесс периодически анализирует статистику по всем таблицам базы данных и определяет необходимость инициации сборки мусора. Выполнение самой очистки делегируется фоновым рабочим процессам `autovacuum workers`. Увеличение максимально допустимого количества рабочих процессов не приводит к пропорциональному ускорению очистки кластера. Все запущенные процессы разделяют между собой единый глобальный лимит на операции ввода-вывода. Увеличение числа параллельных задач приводит к снижению скорости выполнения каждой из них.

Решение о запуске очистки для конкретной таблицы вычисляется с помощью математической формулы. Уравнение складывает фиксированный порог `autovacuum_vacuum_threshold` и процентный множитель `autovacuum_vacuum_scale_factor`, применяемый к актуальному количеству строк. Стандартная конфигурация инициирует очистку при накоплении пятидесяти мёртвых записей плюс двадцати процентов изменённых строк от общего объёма таблицы. Динамическая модель эффективно работает для объектов малого и среднего размера, позволяя системе балансировать интервалы обслуживания.

Применение стандартного процентного множителя к сверхбольшим таблицам вызывает критическую задержку очистки. Для таблицы с одним миллиардом строк базовый коэффициент потребует накопления двухсот миллионов мёртвых записей до момента срабатывания триггера запуска. Обработка сформировавшегося объёма мусора приведёт к гарантированному исчерпанию выделенной оперативной памяти и инициирует ресурсоёмкие многократные проходы по индексным структурам. Эксплуатация крупных таблиц требует принудительного обнуления процентного множителя и установки жёсткого количественного порога срабатывания через системные параметры объекта.

Механизм очистки функционирует на основе системы квотирования ресурсов для минимизации влияния на исполнение пользовательских запросов. Подсистема управления вводом-выводом начисляет рабочим процессам штрафные баллы: `cost_page_hit` за чтение страницы из кеша, `cost_page_miss` за чтение страницы с дискового накопителя и `cost_page_dirty` за запись изменённого блока. При достижении лимита `autovacuum_vacuum_cost_limit` рабочий процесс принудительно приостанавливается на интервал времени, заданный в `autovacuum_vacuum_cost_delay`. Адаптация системы под современные твёрдотельные накопители требует ручной корректировки данных параметров в сторону увеличения лимитов и снижения времени задержки.

При необратимом разбухании файлов базы данных применяется команда полной дефрагментации `VACUUM FULL`. Операция полностью переписывает табличные файлы, устраняя внутреннюю фрагментацию, и возвращает высвобожденное дисковое пространство операционной системе. Команда требует захвата эксклюзивной блокировки, полностью останавливающей любые операции чтения и записи в целевую таблицу. Применение данной команды в эксплуатируемых системах влечёт за собой абсолютный отказ в обслуживании. В качестве отказоустойчивой альтернативы применяются внешние расширения, выполняющие перестроение данных в фоновом режиме на основе механизмов логической репликации и триггеров без длительных эксклюзивных блокировок.

Эффективная эксплуатация подсистемы многоверсионности невозможна без выстроенного мониторинга. База данных предоставляет исчерпывающий набор системных представлений для диагностики состояния хранения. Ключевым инструментом выступает представление `pg_stat_user_tables`, в котором критически важно отслеживать количество мёртвых строк в столбце `n_dead_tup` и время последнего успешного запуска демона очистки в столбце `last_autovacuum`. Для анализа процессов в реальном времени применяется представление `pg_stat_progress_vacuum`, позволяющее определить текущую фазу работы процесса очистки, например сканирование кучи или зачистку индексов, и выявить потенциальные зависания процессов. Точный побайтовый анализ фрагментации файлов таблиц и индексов выполняется с помощью системного расширения `pgstattuple`. Неконтролируемое разбухание таблиц в промышленной среде является следствием отсутствия соответствующих метрик и оповещений в системах мониторинга, а не внутренней недоработкой планировщика базы данных.

Соблюдение ряда архитектурных правил гарантирует стабильную работу высоконагруженных кластеров. Первое правило запрещает полное отключение фоновой очистки в любых сценариях, за исключением ручного восстановления системы в однопользовательском режиме. Второе правило регламентирует настройку крупных таблиц размером в десятки гигабайт и более: для них требуется обязательное принудительное обнуление процентного множителя `autovacuum_vacuum_scale_factor` и управление запуском исключительно через фиксированный количественный порог `autovacuum_vacuum_threshold`. Третье правило требует проактивного резервирования пространства на страницах данных с помощью изменения параметра `fillfactor` на значение 90 для всех таблиц с высокой интенсивностью операций обновления. Это обеспечивает гарантированную работу механизма локальных обновлений без затрагивания индексных структур. Четвёртое правило устанавливает наивысший приоритет для отслеживания системного идентификатора `relfrozenxid`. Приближение этого значения к границе переполнения счётчика транзакций классифицируется как критический инцидент инфраструктуры, требующий немедленного вмешательства.

Архитектура баз данных требует точного расчёта при конфигурации под высокими нагрузками. Каждое изменение системных параметров должно быть математически обосновано, протестировано и строго привязано к пропускной способности физической дисковой подсистемы целевого сервера. Использование конфигурации по умолчанию в масштабах предприятия неизбежно приводит к деградации производительности